



Capstone Track 3: Smart Edge AI with Jetson and IoT Sensors



Objective:

Design and deploy a **real-time AI pipeline at the edge** using **NVIDIA Jetson (Orin/Xavier)** that:

- Captures data from IoT sensors or cameras
- Runs an optimized AI model (via **TensorRT**)
- Performs inference locally or via **Triton**
- Logs outputs and can sync data to the cloud if needed

This project simulates a **real-world smart edge use case** (e.g., smart retail, safety alerting, predictive maintenance).



Step-by-Step Instructions

Step 1: Choose an Edge Use Case

Sample use cases:

- 🛒 Smart retail shelf monitor (detect out-of-stock items)
- 🚒 Industrial safety system (detect PPE or hazards)
- 🏠 Smart home intrusion detection via camera + motion sensor
- 🛠 Predictive maintenance using vibration or temperature sensors

→ Document the scenario, required sensors, and AI logic.

Step 2: Set Up Jetson Environment

Choose your device: **Jetson Orin Nano / AGX Orin / Xavier NX**

1. Flash JetPack OS using [NVIDIA SDK Manager](#)
2. Install dependencies:

```
sudo apt update
sudo apt install python3-pip
pip3 install opencv-python numpy
```

3. Install **DeepStream SDK** (for video) or use **Jetson-optimized Triton container**
-

Step 3: Integrate Sensor or Camera Input

Examples:

- CSI/USB camera via OpenCV
- PIR/motion sensor via GPIO
- Temperature/vibration via I2C or SPI
- MQTT sensor feed via `paho-mqtt`

Sample camera code:

```
import cv2
cap = cv2.VideoCapture(0)
ret, frame = cap.read()
```

For GPIO:

```
import Jetson.GPIO as GPIO
GPIO.setup(pin, GPIO.IN)
```

Step 4: Select and Optimize an AI Model

Choose a pretrained model from:

- [NGC Registry](#)
- TAO Toolkit (e.g., PPE detection, vehicle counting)

Convert to TensorRT:

```
trtexec --onnx=model.onnx --saveEngine=model.engine --fp16
```

Deploy with:

- Pure TensorRT Python API
- OR via **Triton Inference Server**

Step 5: Inference Pipeline on Jetson

Structure:

- Capture data
- Preprocess input
- Run inference (TensorRT engine or Triton)
- Parse output → trigger alerts/log

Example:

```
# Run inference using TensorRT engine
outputs = engine_infer(frame)
if outputs["person_detected"] == True:
```

```
send_alert("Person detected without PPE")
```

Step 6: Edge-Cloud Integration (Optional)

You can:

- Store data locally on device
 - Sync logs to AWS S3 / Azure Blob Storage
 - Stream alerts to MQTT broker / cloud dashboard
 - Use **NVIDIA Fleet Command** or Kubernetes for remote update/deployment
-

Step 7: Containerize the Application

Build a container for Jetson:

```
FROM nvcr.io/nvidia/l4t-m1:r35.2.1-py3
COPY app/ /opt/app/
CMD ["python3", "/opt/app/main.py"]
```

Use docker build and docker run --runtime nvidia

Use **NVIDIA Container Runtime for Jetson**.

Step 8: Add Logging & Monitoring

Log to:

- Local file (CSV , JSON)
- Remote database (InfluxDB, PostgreSQL)
- MQTT dashboard (e.g., Node-RED, Home Assistant)

Example:

```
log = {"timestamp": time.time(), "event": "no_helmet", "confidence": 0.93}
mqtt.publish("factory/alerts", json.dumps(log))
```

Step 9: Final Deliverables

-  **Project Report**
 - Use case
 - Architecture diagram
 - Jetson config + AI model
 - Edge vs cloud tradeoffs

- **Deployment Pipeline**

- Docker container
- Sensor scripts + inference code
- Setup and wiring instructions (if physical)

- **Demo Video**

- Show sensor → AI detection → alert flow
 - Optional: live dashboard updates or cloud sync
-